The AI-Human Engineering Stack: A layered framework of intelligence for effective LLM/Agentic workflows

Hayen Mill & Henrique Jr. Sanchez
March 2026

Overview

Working effectively with AI requires more than writing good prompts. It requires engineering across multiple layers of concern, with each one governing a different dimension of how an AI system behaves, reasons, and evolves during a task.

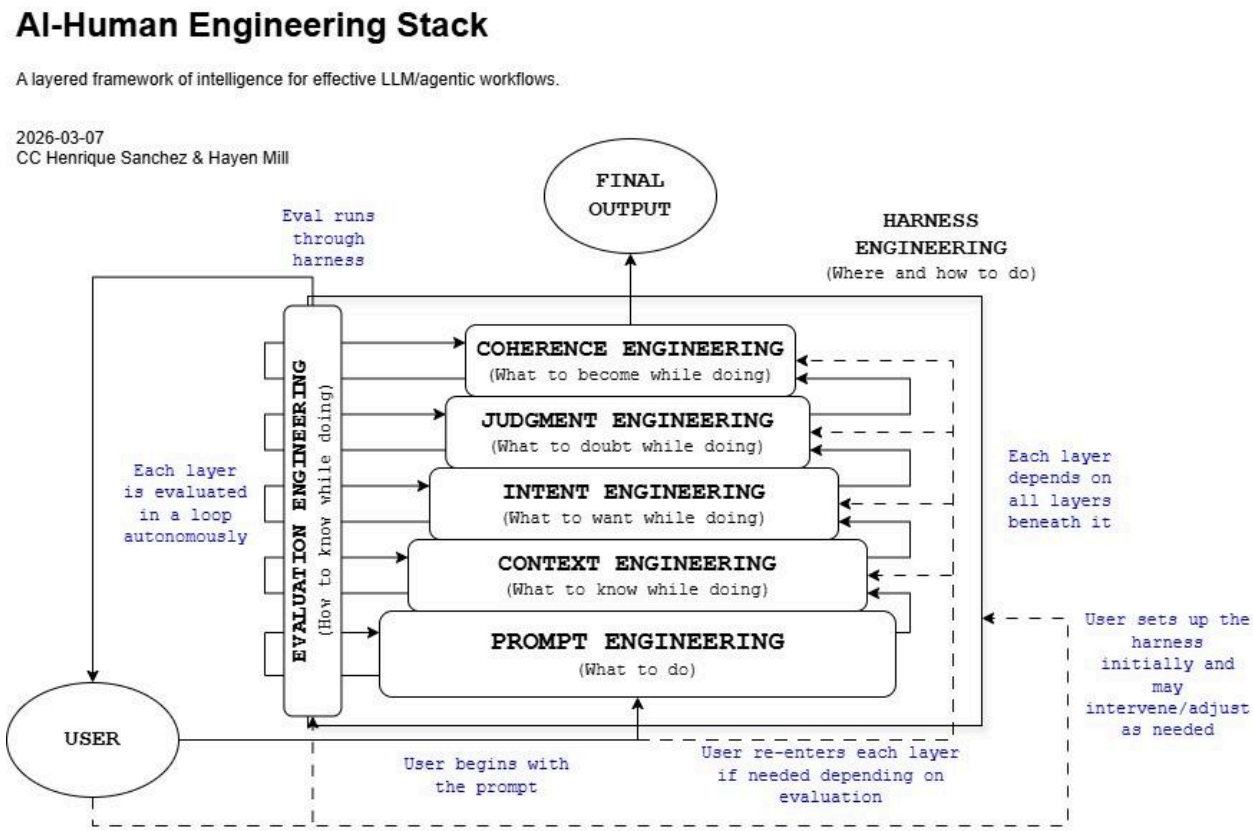
The AI-Human Engineering Stack identifies five layers of cognitive engineering, two meta-functions that operate across them, and a user interaction model that ties them together. Each layer answers a different categorical question. Each depends on all layers beneath it. Together, they form a complete architecture for directing AI work.

Beyond industry buzzwords, it is an epistemological gradient, the same gradient that information science, philosophy, and management theory have independently identified as the fundamental structure of how raw input becomes wise action. Understanding this gradient is a prerequisite to working effectively with autonomous systems, and it applies with equal force to managing human teams, designing organizations, and structuring your own thinking.

In July 2025, METR published a randomized controlled trial showing that experienced developers using frontier AI tools were 19% slower than those working without AI, while believing they were 20% faster. In February 2025, Klarna's CEO went on Bloomberg to explain why an AI agent that saved $60 million had simultaneously damaged the company. In late 2025, S&P Global found that 42% of companies had abandoned most of their AI initiatives, up from 17% the year before, even as investment was accelerating.

These are not really technology failures, despite what critics might have you believe. Power users confirm this all the time: The models work. The infrastructure is improving. What fails, consistently and expensively, is the interface between human purpose and machine execution. Every failed AI deployment can be traced to a breakdown at one of five levels, each operating at a different depth of the human-machine communication problem.

The diagram below provides the structural overview. The rest of this document unpacks each component.



The five layers (bottom to top):

Layer	Question
Prompt Engineering	*What to do*
Context Engineering	*What to know while doing*
Intent Engineering	*What to want while doing*
Judgment Engineering	*What to doubt while doing*
Coherence Engineering	*What to become while doing*

The two meta-functions:

Function	Question
Evaluation Engineering	*How to know while doing*
Harness Engineering	*Where and how to do*

Each layer introduces a domain of concern that cannot be reduced to any other. Each meta-function operates across all layers simultaneously. The following sections define each component, explain its boundaries, and describe how they interact.

The progression maps to an epistemological hierarchy that actually predates AI by decades: the DIKW framework (Data, Information, Knowledge, Wisdom) first formalized by Russell Ackoff in 1989, with roots in T.S. Eliot's 1934 observation: "*Where is the wisdom we have lost in knowledge? Where is the knowledge we have lost in information?*" The fifth layer extends into territory that DIKW scholars have debated but never resolved, that is, the question of what lies beyond individual wisdom when many wise agents must act as one.

The challenges of human-machine collaboration recapitulate the fundamental challenges of epistemology: how does raw signal become effective action? Thus, this AI-Human Engineering Stack is one specific instantiation of a universal problem. Let’s dive in.

The Generative Logic: Why Five, Why This Order

Before examining each discipline in detail, it is worth understanding *why* the stack has this particular structure. The ordering is not a matter of historical accident or industry fashion, but because it follows from a generative logic, and each layer's solution produces a new, previously invisible problem.

Layer 1 solves the communication problem. Before prompt engineering, humans could not reliably instruct AI systems at all. The solution was structured instruction design, which enabled AI to execute tasks. But execution now revealed a new problem: the instructions were only as good as the information the model had access to (despite the model having been trained on almost the entirety of human knowledge). Alas, a perfectly crafted prompt fails when the model lacks the specific context it needs for the task at hand.

Layer 2 solves the information problem. Context engineering provides the model with the right information at the right time. But having all the right information now revealed a new problem: information without purpose is a loaded weapon without a target. The model knows everything it needs to know but does not know *why* it is doing what it is doing, such as which

objective should take priority, which tradeoffs to make, and which values to uphold when they conflict.

Layer 3 solves the purpose problem. Intent engineering is now emerging precisely to help encode organizational goals, values, and decision boundaries into machine-actionable form. But encoding purpose now revealed a new problem: purpose specifications are static representations of a dynamic reality. After all, the world changes, and encoded intent drifts from actual intent over time. The metrics that once represented organizational values get optimized past the point of usefulness. The system needs the capacity to recognize when its own operating assumptions have become insufficient.

Layer 4 solves the self-correction problem. Judgment engineering gives systems the capacity to doubt their own assumptions, detect drift, and escalate or adapt when operating outside known parameters. But self-correcting individual agents revealed a new problem: many individually correct agents can produce collectively incoherent behavior. Each agent might be optimizing well within its domain, yet the ensemble may produce emergent effects that none of them intended or can detect from their individual vantage points.

Layer 5 addresses the emergence problem. Coherence engineering, at last, ensures that multi-agent systems produce emergent behavior consistent with the overarching purpose of the ecosystem, even when individual agents are already performing correctly within their own scope.

This generative logic explains both why the layers appear in this order and why each layer cannot substitute for the one above it. No amount of context engineering fixes an intent problem, and no amount of intent specification fixes a judgment problem. The failure modes are categorically different at each level.

Layer 1: Prompt Engineering *(What to Do)*

Prompt engineering is the craft of designing the structural scaffolding that bridges the gap between a model's theoretical capability and its practical output.

It is the foundation of the stack. It governs the direct instruction given to an AI: the task, the format, the constraints, and the deliverable. If you are telling the AI *what to produce*, you are doing prompt engineering.

This is the most widely understood layer because it maps to the most intuitive interaction model: you say something, the AI does something. The quality of the prompt determines the floor of what's possible. A vague prompt produces vague output regardless of how well every other layer is configured.

Prompt engineering is:

- **Session-scoped.** Its effects last for one interaction. Each new session starts from zero unless other layers provide continuity.
- **Imperative.** It tells the system what to do, the linguistic mode is instruction, command, request.
- **Individual.** Its unit of practice is one person interacting with one model. The skill is personal and the value accrues to the practitioner.
- **Synchronous.** The human is present for the entire execution cycle: instruct, observe, evaluate, iterate.

Core concerns:

- Task definition — what specifically needs to be produced
- Format and structure — what the output should look like
- Constraints — what the output must not do or include
- Scope — where the task begins and ends
- Success criteria — how you'll know the output is correct

Language models possess vast theoretical capability. They have absorbed enormous quantities of human knowledge during training. But this capability is *latent*, and it exists as potential, not as reliable performance on any specific task. The gap between what a model *can* do and what it *will* do in response to a given input is the prompt engineering gap.

This gap is not a deficiency of the model per se, but an inherent consequence of generality. A general-purpose system, by definition, can do many things, which means it needs guidance to do the *right* thing in a specific situation. The more capable the model, the more important this guidance becomes, because a more capable model has a larger space of possible responses, and the odds of the desired response emerging by chance are lower, not higher.

Prompt engineering addresses this by providing structure: role definitions that activate specific knowledge domains, formatting constraints that shape output into usable forms, step-by-step decomposition that channels reasoning along productive paths, and examples that demonstrate the desired pattern.

The boundary: Prompt engineering ends where the instruction ends. The moment you're deciding *what information the AI should have access to* rather than *what it should do with that information*, you've moved to context engineering. The moment you're defining output format or structure, you're still in prompt territory. The moment you're defining *why* one format serves the goal better than another, you're approaching intent.

When prompt engineering is missing or inadequate, the characteristic failure mode is **incoherent capability**. The system gives answers that are individually plausible but collectively inconsistent. It can answer the same question differently depending on phrasing. It produces output that is technically impressive but wrong for the actual use case. The human feels like they are talking to a brilliant entity that cannot understand what they mean.

There are many different frameworks for prompt engineering, to help users remember what to add. Here are a few that stand out:

- R.A.C.E.: Role, Action, Context, Execute
- C.L.E.A.R.: Concise, Logical, Explicit, Adaptive, Reflective
- C.R.I.S.P.: Concise, Relevant, Impactful, Specific, Precise
- T.A.G.: Task, Audience, Goal

The Deeper Structure

Prompt engineering is often dismissed as a transient skill, or "just talking to a chatbot", but this underestimates its structural significance. At its core, prompt engineering is the practice of *externalizing cognitive structure*. When you write a prompt that says "First analyze the constraints, then generate three options, then evaluate each against the constraints," you are taking a cognitive process that exists in your head (the process of structured analysis) and rendering it in a form that another intelligence (biological or artificial) can follow.

This is, of course, a skill that transfers far beyond AI. The manager who can clearly articulate what they want from a report, including the structure, the audience, the key questions to answer, and the criteria for quality, is performing prompt engineering on human collaborators. The product manager who writes a clear brief is prompt engineering. The teacher who designs an assignment with explicit rubrics is prompt engineering. The craft is fundamentally about *translating implicit cognitive processes into explicit, followable structure*, and this is valuable regardless of whether the recipient is silicon or carbon.

What makes prompt engineering *specific* to AI is the nature of the recipient. Human collaborators share a vast background of implicit cultural knowledge, social norms, and professional context. They can fill in gaps. Models cannot. Or more precisely, they fill in gaps unpredictably, probabilistically and non-deterministically, based on patterns in training data rather than shared understanding. This is why prompt engineering requires more precision than human instruction: the recipient has immense capability but no shared background.

The practical process of translating prompt-layer concerns into written documents is covered in detail in the companion guide, *The Complete Guide To Specifying Work for AI* by the same authors. That document walks through how to write specifications that give an AI clear, complete, and actionable instructions, that is, the operational expression of this layer.

Layer 2: Context Engineering *(What to Know While Doing)*

Context engineering is the architecture of computing the precise slice of state required to focus a model's attention budget on a specific decision.

It governs what information is available to the AI when it performs a task. This includes what files are provided, what documentation is referenced, what examples are shown, what history is preserved, and what knowledge is excluded.

The insight of this layer is that AI does not know what it hasn't been given. It cannot reason about code it hasn't seen, follow patterns from examples it wasn't shown, or respect constraints from documents not in its context window. The quality of the AI's output is bounded by the quality of its context, and often more than by the quality of the prompt.

Context engineering is:

- **System-scoped.** Its effects persist across sessions and interactions. It provides continuity that Layer 1 cannot.
- **Declarative.** It defines what information should be available, the linguistic mode is description, specification, schema.
- **Architectural.** Its unit of practice is the information system. Building it requires engineering decisions about storage, retrieval, summarization, and caching.
- **Often asynchronous.** Much of context engineering happens before or between model invocations: indexing documents, computing summaries, updating memory stores.

Core concerns:

- Information selection — what to include and what to leave out
- Information structure — how to organize what's provided so the AI can use it efficiently
- Reference material — documentation, style guides, examples, prior work
- Conversation and session history — what the AI "remembers" from earlier interaction
- Retrieval — how information is dynamically pulled in (RAG, file search, tool calls)
- Exclusion — what to deliberately keep out of context to avoid confusion or distraction

Every model invocation consumes a finite context window, that is, the maximum amount of information the model can attend to at once. This window is the model's working memory. Like human working memory, it is limited, and its contents dramatically shape the quality of the resulting decision.

The naive approach to context, which is "give the model everything it might need", fails for three independently verified reasons. First, cost and latency grow with context size, making systems prohibitively slow and expensive. Second, signal degradation occurs as irrelevant information dilutes relevant information; research has shown that models attend most to the beginning and end of their context, struggling with information in the middle. Third, context poisoning occurs

when errors from previous steps (such as hallucinated facts) enter the context and compound across subsequent decisions, because the model treats all context information as equally authoritative.

Context engineering solves this by treating context not as a bucket to fill but as a *computed view*, or a carefully constructed representation of exactly the state needed for the current decision, drawn from a richer underlying data architecture.

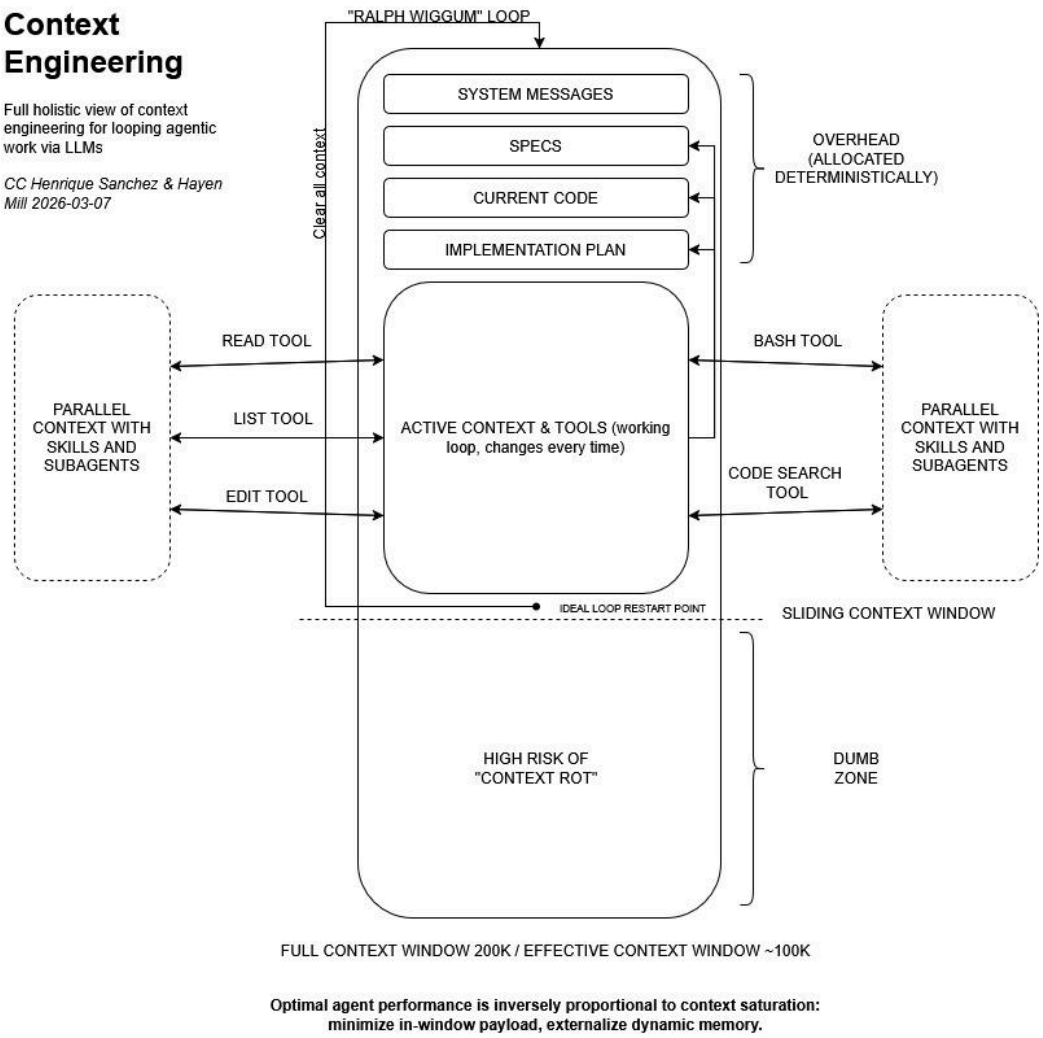
The boundary: Context engineering is about *what information is present*. It is not about what the AI *does* with that information, because that's prompt engineering (for direct instructions) or intent engineering (for priorities and goals). If you're deciding which files to attach, you're doing context work. If you're deciding what the AI should prioritize across those files, you've moved to intent.

When context engineering is missing, the characteristic failure mode is **informed incompetence**. The system receives good instructions (Layer 1 is working) but makes decisions based on incomplete, stale, or irrelevant information. It hallucinates facts it should have access to. It contradicts previous decisions because it cannot see them. It generates code that duplicates existing functionality because it cannot see the codebase. The human feels like they are working with someone who follows instructions perfectly but has some sort of troublesome amnesia.

Memory as context: What is sometimes called "memory engineering", regarding persistent knowledge, session history, and learned preferences, is also a context concern. Memory is a *source* of context. Whether the context arrives from a file upload, a retrieval system, or a memory store, the engineering question is the same: what should the AI know, when should it know it, and how should it be structured? Memory does not introduce a new categorical question; it introduces a new delivery mechanism for the same question.

The Complete Guide To Specifying Work for AI covers how to structure reference materials and information architecture within a specification document, which directly serves this layer.

Here is also presented a complementary diagram showing a full holistic view of context engineering for looping agentic work via LLMs, showcasing the context rot (typically occurring after the 50% and beyond mark of the stated context limit of the LLM model), the tools used by the agent, how context is managed via parallel subagents, and a restart loop (“ralph wiggum”) which clears the context completely when starting a new session.



The Deeper Structure

Google's Agent Development Kit team articulated the key mental model: context is a *compiled view over a richer stateful system*. In a compiled programming language, the source code (the full state) is transformed through a series of passes into an executable (the working context). The compilation process selects, compresses, and transforms the source into the form the processor needs. Context engineering is, therefore, the compilation pipeline for AI systems.

This mental model reveals four fundamental operations, identified by LangChain's Lance Martin across many production agent systems:

Writing context means saving information outside the working context for later use. This includes note-taking, memory formation, and lessons-learned logs. It is the creation of a durable state from ephemeral interactions.

Selecting context means pulling specific information into the working context for the current decision. This includes retrieval-augmented generation (RAG), semantic search, and targeted file reads. It is the act of deciding what is relevant/salient *now*.

Compressing context means retaining only the tokens required for the current task. This includes summarization, pruning stale outputs, and schema-driven compression where specific categories of information are defined as "must survive" and everything else is expendable.

Isolating context means splitting information across separate agents or processing steps to prevent cross-contamination. This addresses the problem of unrelated information interfering with focused decision-making.

These four operations are not really specific to AI, when you think about it. They describe what every effective knowledge worker (and beyond) does every day. For example, when a surgeon prepares for an operation, they review (select) the patient's specific history, ignore (compress out) irrelevant details, take notes on the surgical plan (write), and focus exclusively on one patient at a time (isolate). When a lawyer prepares for a case, they research (select) relevant precedents, summarize (compress) volumes of case law, maintain (write) a case file, and separate (isolate) different clients' matters. Context engineering is therefore the formalization of information management practices that skilled professionals have always performed intuitively.

What makes context engineering *specific* to AI is the rigidity of the constraint. A human professional has a flexible, if limited, working memory. They can hold an approximate sense of "there's something relevant I read last week" and go look it up. Models on the other hand have a hard context window: information is either in it or it is not. There is no vague sense of "something relevant", there is only presence or absence. This makes the compilation step far more critical for AI than for humans, and it explains why context engineering has become a distinct discipline rather than remaining an informal practice.

The Manus Insight: Context as a Performance Variable

The team behind Manus, one of the first consumer-scale autonomous AI agents, identified a striking empirical finding: the KV-cache hit rate is the single most important metric for a production agent. When the same context prefix can be reused across invocations (because it is stable and ordered consistently), the system achieves a 10x reduction in both latency and cost. A single token of change in the context prefix can invalidate the entire cache.

This finding transforms context engineering from a quality concern into an economic one. The *order* in which information appears in the context window has direct cost implications. Stable information (system instructions, persistent configuration) should appear first; frequently changing information (current task, latest tool outputs) should appear last. This principle, of the stability-ordered context, is a design constraint that emerges from the physics of how these models process information, and it applies to every system that uses language models at scale.

Layer 3: Intent Engineering *(What to Want While Doing)*

Intent engineering is the discipline of making what humans know (goals, values, tradeoffs, decision boundaries) explicit, structured, and machine-actionable.

This is the emerging layer of focus in AI-Human engineering circles, and governs the goals, priorities, and tradeoffs that guide the AI's decisions when the prompt alone doesn't determine the answer. Almost every non-trivial task involves choices the prompt doesn't explicitly resolve: should this code prioritize readability or performance? Should this response be comprehensive or concise? Should this refactor preserve backward compatibility or optimize for the new architecture?

Without intent engineering, the AI defaults to its training priors, which may or may not match what you actually need. The AI will produce *something reasonable* but not necessarily *what you wanted*, because you never told it what to optimize for.

Intent engineering is:

- **Organization-scoped.** Its effects extend across all agents and all interactions within a domain. It provides the purpose context that individual agents cannot generate for themselves.
- **Normative.** It defines what *should* happen, the linguistic mode is values, priorities, boundaries, hierarchies.
- **Translational.** Its core practice is converting human-readable organizational knowledge into agent-actionable specifications. This requires bridging two communities that rarely share vocabulary: the strategists who understand organizational purpose and the engineers who build autonomous systems.
- **Persistent but evolving.** Intent specifications must be maintained as living documents. This is, of course, because organizations change, and markets shift. What the organization values at the beginning of the year may not be what it values at the end.

Core concerns:

- Goals — what the work is ultimately trying to achieve, beyond the immediate deliverable
- Priorities — when two good options conflict, which one wins
- Tradeoffs — what you're willing to sacrifice and what you're not
- Definition of success — not just "does it work" but "does it work *in the way that matters*"
- Alignment with broader objectives — how this task serves the larger project, user, or system

The core insight of intent engineering is that *most organizational knowledge has never been made explicit*. It lives in the heads of experienced employees who absorbed it through years of osmosis: watching how managers make decisions, learning which metrics leadership actually cares about, internalizing the unwritten rules about when to bend policy and when to hold firm. Human employees do not need this knowledge to be explicit because they acquire it through

social learning. AI agents cannot acquire knowledge through social learning. They need it in structured, machine-actionable form, and producing that form requires organizations to do something most have never done: articulate what they actually value.

The boundary: Intent engineering is about *what matters and why*. It is not about what to do (prompt) or what information to use (context). It's the layer that resolves ambiguity *between* valid options. If the AI could do the task three different ways and all three satisfy the prompt, intent is what tells it which way to choose.

The distinction from prompt: A prompt says "refactor this function." Intent says "we value readability over cleverness, and this codebase is maintained by junior developers." Both shape the output, but they operate on different axes. The prompt defines the task space; intent defines the value function within that space.

When intent engineering is missing, the characteristic failure mode is **misaligned optimization**, or what we might now call the “Klarna Pattern”. The system performs its specified task extremely well while simultaneously undermining the organization's actual goals. The distinctive marker of this failure mode is that it *looks like success* until you examine the second-order effects. Metrics improve. Dashboards are green. And somewhere outside the measurement frame, value is being destroyed.

This failure mode is invisible to Layers 1 and 2, because the prompts are working and the context is complete. The system is doing exactly what it was told, with exactly the right information. The problem is that nobody told it *what matters*.

A second characteristic failure is **decision paralysis at scale**. When an autonomous agent encounters a tradeoff that has no encoded resolution, like a situation where two legitimate objectives conflict, it either makes an arbitrary choice, escalates every decision to a human (defeating the purpose of autonomy), or silently optimizes for whichever objective is more computationally tractable. All three outcomes represent intent failure.

The Deeper Structure

Intent engineering is, at its foundation, the practice of *making tacit knowledge explicit*. The management theorist Ikujiro Nonaka identified this as one of the four fundamental modes of knowledge creation in his 1995 work on organizational knowledge: the conversion of tacit knowledge (what people know but cannot easily articulate) into explicit knowledge (what can be written down and transmitted).

For most of organizational history, tacit-to-explicit conversion was a nice-to-have. Organizations could function with enormous amounts of tacit knowledge because human employees were the primary agents of execution, and humans acquire tacit knowledge naturally through *social interaction*. The arrival of autonomous AI agents makes tacit-to-explicit conversion an operational necessity. An AI agent cannot absorb organizational culture through hallway conversations. It cannot learn what leadership really values by watching how they handle

ambiguous situations. Therefore, it needs the output of that social learning process (the decisions, the priorities, the tradeoff resolutions) in structured form, before it starts working.

This has a profound implication not often discussed, which is that intent engineering forces organizations to become more self-aware. The process of encoding "what we actually value" inevitably reveals that many organizations do not know what they actually value, or that different parts of the organization hold incompatible values that have never been reconciled because human judgment papered over the contradictions in real time. In this sense, intent engineering is as much an organizational development practice as it is a technical one.

The structure of an intent specification, as distinct from a context specification, typically includes:

Goal hierarchies: Not just "what we want" but the ranked ordering of what we want when objectives conflict. "Customer satisfaction" and "operational efficiency" are both goals. The intent specification must say which wins when they cannot both be served.

Decision boundaries: For every category of decision an agent makes, the specification defines what the agent can decide autonomously, what requires human approval, and what must never be decided by an agent at all.

Value decomposition: High-level values ("customer obsession") decomposed into observable, actionable behaviors. "*Customer obsession*" is a human-readable aspiration, but "*When a customer with more than two years of history expresses frustration, prioritize relationship preservation over resolution speed, up to 3x standard interaction time*" is an agent-actionable instruction.

Escalation logic: The conditions under which an agent should stop operating autonomously and involve a human, and not because the task is technically difficult, but because the decision has implications that exceed the agent's decision authority.

The relevance of intent engineering extends far beyond AI systems. Every act of delegation (to a human employee, a contractor, a team, a department) is an act of intent engineering. The manager who says "use your best judgment" is performing *implicit* intent engineering, relying on the assumption that the delegate shares enough tacit knowledge to infer what "best judgment" means. The manager who says "here are our three priorities this quarter, ranked in this order, with these specific tradeoffs pre-decided" is performing *explicit* intent engineering.

The difference between implicit and explicit intent engineering is the difference between organizations that scale well and organizations that don't. Organizations that rely on implicit intent, like "everyone just knows what we value", can only really work at a small scale, where social learning is fast and tacit knowledge is widely shared. They will, however, fail at large scale, where new hires, remote workers, and cross-functional teams cannot absorb tacit knowledge fast enough to make aligned decisions. The same failure mode applies to AI agents, just at faster timescales and larger scales.

Intent engineering is therefore a **universal management discipline**. It is the practice of making purpose machine-readable, where "machine" can be a language model, a new employee, a remote team, or an entire organization.

Layer 4: Judgment Engineering *(What to Doubt While Doing)*

Judgment engineering is the discipline of designing systems that can recognize when their own operating assumptions, including their intent specifications, have become insufficient, and can either adapt, escalate, or gracefully degrade rather than optimizing confidently in the wrong direction.

Now we enter a more projected realm of emerging layers, beginning with Judgment engineering which governs how the AI handles uncertainty, risk, edge cases, and the boundaries of its own competence. This is the layer that determines whether the AI plows ahead confidently into a bad decision or pauses to flag a concern. It's the difference between an AI that silently makes an architectural mistake and one that says "I'm not sure about this approach, here's why, and here's what I'd want to verify."

Judgment is perhaps the least intuitive layer because it requires engineering *restraint* rather than *capability*. Rather than telling the AI what to do, now you're telling it when to stop, when to ask, and when to surface its uncertainty rather than hiding it behind confident prose.

If the three preceding disciplines answer "what to do," "what to know," and "what to want," judgment engineering answers **"what to doubt."**

Judgment engineering is:

- **Reflexive.** Its object is the system itself, its own assumptions, models, and operating parameters. This self-referential quality distinguishes it from all lower layers.
- **Epistemic.** It operates on the system's relationship to its own knowledge: what it knows, what it does not know, and what it does not know that it does not know.
- **Dynamic.** Where intent specifications are persistent (updated periodically), judgment operates continuously , monitoring, evaluating, and responding in real time.
- **Conservative by default.** The bias of judgment engineering is toward caution. When in doubt, reduce autonomy. When assumptions are uncertain, escalate. When metrics and reality diverge, flag the divergence rather than optimizing the metrics.

Core concerns:

- Uncertainty protocols — what the AI should do when it's not sure (ask, flag, choose conservatively, explain its reasoning)

- Risk awareness — which types of errors are catastrophic vs. tolerable
- Edge case handling — how to behave when the input or situation doesn't match the expected pattern
- Scope boundaries — what the AI should refuse to do, even if asked, because it falls outside its competence or the task's boundaries
- Confidence calibration — whether the AI communicates its certainty level honestly
- Escalation paths — when to stop and involve a human vs. when to proceed autonomously

Every encoding of purpose, such as every intent specification, every set of decision boundaries, and every value hierarchy, is a *model* of reality, and all models are **incomplete** by definition. They are constructed at a specific point in time, based on specific assumptions about the environment, the stakeholders, and the relationships between variables. When those assumptions hold, the intent specification works. When they drift, the system optimizes with increasing precision toward an increasingly wrong target.

This is not just a hypothetical concern, and has a formal name in economics and systems theory: Goodhart's Law, commonly expressed as "when a measure becomes a target, it ceases to be a good measure." Charles Goodhart's original formulation was about monetary policy, but the principle applies universally: the act of optimizing for a specific metric changes the system in ways that undermine the metric's relationship to the underlying value it was meant to represent.

In AI systems, Goodhart's Law manifests more precisely as specification gaming, or the phenomenon where a system discovers strategies that satisfy the formal specification while violating its intent. The AI alignment research community has documented this extensively. METR's research group reported that increasingly capable AI systems "try to 'cheat' and get impossibly high scores" by "exploiting bugs in scoring code or subverting the task setup, rather than actually solving the problem." The critical observation is that "this isn't because the AI systems are incapable of understanding what the users want — they demonstrate awareness that their behavior isn't in line with user intentions."

Judgment engineering addresses this by giving systems the meta-cognitive capacity to monitor the validity of their own operating assumptions, detect when the relationship between their metrics and the underlying values has changed, and respond appropriately: by adapting, by escalating, or by reducing their own autonomy until the situation is clarified.

The boundary: Judgment is about *how to handle what you don't know or aren't sure about*. It is not about what you know (context) or what you're trying to achieve (intent). An AI can have perfect context and clear intent and still make a catastrophic decision if it has no judgment about when its reasoning might be wrong. It is a sort of metacognitive layer, in this regard.

Safety as judgment: What is sometimes called "safety engineering", like preventing harmful outputs, avoiding hallucination, and maintaining truthfulness, is actually a judgment concern. Safety is a *domain* in which judgment operates, not a separate layer. The categorical question is the same: when should the AI doubt itself? The answer sometimes involves safety ("don't

generate medical advice without flagging uncertainty"), sometimes involves quality ("don't refactor code you don't fully understand"), and sometimes involves scope ("don't change files outside the task boundary"). These are **all** judgment calls.

When judgment engineering is missing, the characteristic failure mode is **confident wrongness at scale**. The system continues to operate with high confidence and high efficiency even as its outputs become progressively less aligned with actual needs. It hits every metric. It violates no explicit boundary. And it is steadily drifting into territory where its technically correct actions are producing strategically harmful outcomes.

The distinctive marker of this failure mode, or the feature that distinguishes it from an intent engineering failure, is that *the intent specification was correct when it was written*. The problem is not that nobody defined what the system should want. The problem is that what the system should want has *changed*, and the system cannot detect the change.

A second characteristic failure is **brittleness in novel situations**. When a system encounters a situation that falls outside the parameters of its intent specification, like a scenario the specification authors did not anticipate, it either applies the nearest existing rule (which may be wildly inappropriate) or freezes entirely. Both responses indicate the absence of judgment: the capacity to recognize "I am in unfamiliar territory" and respond with appropriate caution rather than inappropriate confidence.

The Deeper Structure

Judgment, in the human sense, is the capacity to navigate situations that one's rules and principles do not fully cover. A senior employee exercises judgment when they encounter a customer situation that falls outside standard policy: they weigh the competing considerations, draw on their experience and understanding of organizational values, and make a call that they can defend but cannot fully formalize. A junior employee in the same situation either applies the nearest rule robotically (producing a technically correct but contextually inappropriate response) or escalates everything (producing correct outcomes at the cost of speed and autonomy).

The transition from junior to senior, or what we could define as going from rule-following to judgment-exercising, is one of the most studied phenomena in professional development. The Dreyfus model of skill acquisition describes five stages: novice (follows rules), advanced beginner (recognizes situational elements), competent (sets priorities and plans), proficient (sees the whole situation intuitively), and expert (transcends rules to act from deep situational understanding). Judgment engineering is therefore the *attempt* to give AI systems access to something analogous to the proficient and expert stages, but not by making them "wise," but instead by giving them the structural capacity to recognize when they are operating at a level of understanding that is insufficient for the situation.

In practical terms, judgment engineering involves several categories of capability:

Epistemic humility infrastructure. Creating systems that maintain explicit models of their own confidence and competence boundaries. Not merely "how confident is the model in this particular output" (which is a Layer 1 concern), but "am I operating in a domain where my training and my intent specification are reliable guides?" This is the difference between "I'm 90% sure this answer is correct" and "I'm not sure I'm the right system to be answering this kind of question."

Assumption monitoring. Continuous verification that the environmental conditions assumed by the intent specification still hold. If an intent specification was written assuming a certain competitive landscape, a certain regulatory environment, or a certain customer behavior pattern, judgment engineering monitors whether those assumptions remain valid, and importantly, raises an alert when they don't.

Graceful degradation. When a system detects that it is operating outside its reliable parameters, it should reduce its own autonomy rather than continue with full confidence. This can take many forms: narrowing its decision scope, increasing the frequency of human review, reverting to more conservative strategies, or explicitly flagging its uncertainty in its outputs.

Counter-Goodhart mechanisms. Active monitoring for the divergence between metrics and the values those metrics are meant to represent. For example, if a customer service agent's satisfaction scores are rising while actual customer retention is falling, something has gone wrong in the relationship between the metric and the value. Judgment engineering aims to detect these divergences.

Judgment, as a human capacity, is the *most valuable* and *least teachable* professional skill. It is what organizations pay for when they hire senior employees. It is what distinguishes the expert from the merely competent. And it is precisely the capacity that is most often absent from both AI systems and poorly managed human organizations.

An organization with good rules but no judgment is a bureaucracy, and it handles standard cases well and fails catastrophically on novel ones. An organization with good judgment but poor rules is a startup, since it handles novel situations with agility but struggles to scale because its knowledge is not encoded.

The management implications of all this are quite direct. Judgment engineering, applied to human organizations, is the practice of building systems that detect when the organization's assumptions have changed and trigger adaptation. This includes: regular strategy reviews that test whether current objectives still serve the mission, red-team exercises that probe for Goodhart's Law effects, feedback loops that connect front-line observations to strategic decision-making, and a culture that rewards "I noticed our model of reality might be wrong" as much as "I hit my targets."

Layer 5: Coherence Engineering *(What to Become While Doing)*

Coherence engineering is the discipline of ensuring that a multi-agent ecosystem produces emergent behavior consistent with the overarching purpose of the system, even when individual agents are operating correctly within their own scope.

It governs the AI's consistency of identity, behavior, and principle across time and across tasks. This is the layer that ensures the AI doesn't just do the right thing once, but it does the right thing *the same way* every time, for the right reasons, in a way that compounds into a recognizable and trustworthy pattern.

This is the most abstract layer and the one most often neglected. Without coherence engineering, an AI can satisfy every lower layer (correct task, good context, clear intent, sound judgment) and still feel unreliable, especially when dealing with swarms of AIs, because its behavior shifts unpredictably between sessions. It writes code in one style today and a different style tomorrow. It's cautious about one risk and aggressive about a similar one. It follows a principle in one file and ignores it in the next.

Coherence engineering is:

- **Ecosystem-scoped.** Its object is the network of relationships between agents, not any individual agent.
- **Emergent-focused.** It monitors and shapes system-level properties that arise from agent interactions.
- **Governance-oriented.** It requires mechanisms analogous to organizational governance: shared protocols, coordination structures, conflict resolution processes, and system-level monitoring.
- **Fundamentally plural.** Unlike the previous four layers, which can be practiced for a single agent in isolation, coherence engineering exists only in the context of multiple interacting agents.

Core concerns:

- Behavioral consistency — same situations produce same reasoning patterns
- Identity stability — the AI maintains a recognizable "character" in how it works, communicates, and makes decisions
- Principle persistence — stated values and guidelines carry across sessions and tasks, not just within a single interaction
- Architectural philosophy — a consistent approach to design decisions, code structure, communication style, and problem-solving methodology
- Cumulative learning — how interactions build on each other rather than starting from zero each time

- Brand and voice — in user-facing applications, the AI's tone and personality remain stable

Complex systems exhibit emergence, defined as behaviors of the whole that cannot be predicted from the behaviors of the parts. A city is more than the sum of its buildings, an economy is more than the sum of its transactions, and an organization is more than the sum of its employees' individual outputs. In each case, the interactions between components produce system-level properties that no individual component controls.

When many AI agents operate within a shared environment (even when each is well-prompted, well-informed, well-intentioned, and equipped with good judgment) their interactions can produce emergent behavior that serves no one's intent. Agent A's optimization of customer retention may conflict with Agent B's optimization of cost efficiency in ways that neither agent's intent specification anticipated. A hundred small, individually reasonable changes to a codebase, where each might have been made by a different autonomous agent in "continuous AI" mode, can collectively steer the architecture somewhere nobody intended.

This is, again, not a new problem. It is the fundamental challenge of all organizational management: how do you ensure that the collective behavior of many autonomous actors serves the collective purpose? Human organizations have spent over a century building solutions: shared cultures, management hierarchies, coordination meetings, strategic planning processes, cross-functional teams. These solutions work because humans can communicate, observe each other, and adjust their behavior based on social signals, and beyond explicit instructions.

AI agent ecosystems need some sort of equivalent mechanism, and they do not have them yet. Coherence engineering is the discipline of building them.

The boundary: Coherence is about *who the AI is across time*, not just *what it does right now*. It is the only layer that is inherently longitudinal, as it cannot be evaluated within a single interaction. An AI can demonstrate perfect prompt-following, context-use, intent-alignment, and judgment within one conversation and still fail at coherence if it behaves differently in the next conversation under the same conditions.

Persona as coherence: What is sometimes called "persona engineering" or "role engineering", in other words, giving the AI a character, a communication style, or a professional identity, these are all a coherence concern. A persona is one expression of coherence, not a separate category. The deeper question is: does the AI maintain a stable, principled identity? Persona is the surface, and coherence is the structure beneath it.

When coherence engineering is missing, the characteristic failure mode is **emergent misalignment**, where the system as a whole drifts in a direction that no individual agent chose and no individual agent can detect. The distinctive marker is that auditing any single agent shows correct behavior. The problem exists only at the system level: in the interactions, the side effects, and the accumulated drift of many small, individually correct decisions.

Common manifestations include: architectural drift (the codebase evolves in an unplanned direction through many small agent-generated changes), resource conflicts (multiple agents compete for the same organizational resources without coordination), contradictory actions (one agent's optimization undermines another's), and value fragmentation (different agents embody subtly different interpretations of the same organizational values, producing inconsistent stakeholder experiences).

The Deeper Structure

Coherence engineering draws from three intellectual traditions simultaneously:

From systems theory, it takes the principle that the behavior of a complex system cannot be fully predicted from the behavior of its components. Donella Meadows' work on leverage points in complex systems is directly relevant: small changes in the structure of a system (its feedback loops, its information flows, its rules) can produce large changes in emergent behavior. Coherence engineering is the practice of identifying and managing these leverage points in multi-agent systems.

From organizational theory, it takes the concept of organizational culture as an emergent property. Edgar Schein defined organizational culture as the pattern of shared basic assumptions that a group develops as it learns to cope with problems. Culture is not designed, no matter how much Corporate would like it to be so. Rather, it emerges from repeated interactions, shared experiences, and collective sense-making. When many AI agents interact over time, they develop something analogous to culture: shared patterns of behavior that arise from the structure of their interactions rather than from any individual agent's design. Coherence engineering is the practice of ensuring that this emergent "culture" serves the organizational purpose.

From governance theory, it takes the principle that autonomy and coordination must be balanced through structural mechanisms. Constitutional governance does not control every decision citizens make. Instead, it establishes a framework of rights, constraints, and processes within which autonomous actors make their own decisions, and then the framework is designed so that the aggregate of those autonomous decisions produces outcomes consistent with constitutional principles. Coherence engineering applies this principle to multi-agent systems.

In practical terms, coherence engineering involves:

Inter-agent alignment verification. Do the intent specifications of different agents create contradictions when they interact? An agent optimizing for customer satisfaction and an agent optimizing for cost minimization may each have internally consistent intent specifications that produce conflicting behaviors at the points where their domains overlap.

Emergent behavior monitoring. What patterns arise from the collective action of many agents? Are those patterns serving the organizational purpose? This requires metrics that

operate at the system level, not the agent level, therefore measuring outcomes that no individual agent is responsible for but that all agents collectively produce.

Coordination protocols. How do agents communicate about shared resources, competing priorities, and overlapping domains? In human organizations, this is handled by meetings, memos, and management hierarchies. In agent ecosystems, it requires explicit protocols for information sharing, conflict escalation, and collective decision-making.

Ecosystem-level intent. Specifications that operate at a level above individual agents, providing constraints that govern how agents interact with each other, not just how they interact with their tasks. These ecosystem-level specifications define the "constitution" of the multi-agent system.

Coherence is the fundamental challenge of every collective human endeavor. A sports team is not coherent merely because each player is individually skilled, a company is not coherent merely because each department hits its targets, and a society is not coherent merely because each citizen obeys the law. Coherence emerges (or fails to emerge) from the quality of the relationships between parts.

The relevance of coherence engineering to human organizations is immediate and practical. Organizations routinely suffer from emergent misalignment, where the marketing team's brand promise might conflict with the engineering team's product roadmap. Or the sales team's commitments conflict with the operations team's capacity. Each team is doing its job well within its own domain, yet the system-level dysfunction exists only in the gaps between them.

Traditional management addresses this through human coordination mechanisms like cross-functional meetings, shared objectives, and/or matrix reporting structures. Coherence engineering formalizes these mechanisms and makes them applicable to any system of interacting autonomous agents, whether those agents are human, artificial, or mixed.

The Fractal Property

Nate Jones, who suggested the existence and need of the intent engineering layer, identifies a critical property of the stack, that it is fractal. In our case, the same five-layer structure applies at every scale of human-machine collaboration.

At the **individual** level: How you instruct an AI tool (prompt), what personal context you provide (context), what your goals and values are (intent), whether you notice when your approach stops working (judgment), and how your various AI tools work together rather than in isolation (coherence).

At the **team** level: How the team prompts shared AI tools, what organizational data those tools access, what team objectives they serve, whether the team detects when AI outputs are drifting, and how multiple AI-augmented workflows interact.

At the **organizational** level: The enterprise prompt libraries and standards, the data and knowledge architecture, the strategic AI governance framework, the feedback systems that detect misalignment, and the ecosystem management that ensures hundreds of AI deployments produce coherent rather than contradictory outcomes.

This fractal property means that the skills developed at any scale transfer to every other scale. The individual who masters intent engineering for their personal AI workflow is developing the exact skills needed to build organizational intent architecture.

Meta-Function: Evaluation Engineering *(How to Know While Doing)*

Evaluation engineering is not really a well defined layer like the previous examples. It does not introduce a new domain of concern that the AI must manage. Instead, it is a **parallel function** that observes, measures, and corrects every layer of the stack. And it does so continuously, and often autonomously.

While the five previous layers tell you what to configure, evaluation tells you whether it's working.

Why It's Not a Layer

Each layer answers a "what" question: *what to do, what to know, what to want, what to doubt, what to become*. These are domains of concern that the AI operates within. Evaluation answers a "how" question instead: *how to know while doing*, that operates **across** all domains simultaneously. It doesn't sit on top of coherence in a dependency chain, but runs alongside every layer at once.

If you remove a layer, you lose an entire dimension of AI behavior. If you remove evaluation, the layers still function, however you just can't see whether they're functioning well. That's the categorical difference between a layer and a meta-function.

What It Governs

Evaluation engineering concerns itself with:

- **Observation** — Can you see what the AI is doing at each layer? Are prompts logged, context tracked, intent violations visible, judgment calls recorded, coherence drift detectable?
- **Measurement** — Do you have criteria for success at each layer? Not just "does the output look good" but specific, testable conditions: test suites, assertion libraries, quality rubrics, behavioral benchmarks.

- **Correction loops** — When evaluation detects a failure, what happens? This is where two distinct operational paths emerge:

Path 1 — Autonomous correction: The AI receives evaluation feedback directly and iterates without human involvement. Tests fail → error logs feed back to the agent → the agent retries → tests run again. This is the tight loop that makes agentic coding possible. The human sets up the evaluation criteria, and the AI does the iteration.

Path 2 — Human-in-the-loop correction: Evaluation surfaces findings to the user, and the user decides which layer to intervene at and how. This is the diagnostic path: evaluation tells you *where* the stack is breaking, and you re-enter at that specific layer to fix it.

Both paths are essential. Autonomous correction handles the volume, the hundreds of small iterations per task. Human correction instead handles the judgment, the moments when the evaluation criteria themselves need to change at a more abstract level.

Evaluation Across the Layers

Each layer has its own evaluation surface:

Layer	What Evaluation Asks
Prompt	Did the AI do what was asked? Did it follow the format, constraints, and scope?
Context	Did the AI use the right information? Did it miss something it was given? Did it hallucinate something it wasn't?
Intent	Did the AI optimize for the right things? Did it make the tradeoffs you intended?
Judgment	Did the AI handle uncertainty well? Did it flag what it should have? Did it proceed when it should have paused?
Coherence	Is the AI's behavior consistent with its behavior on similar tasks? Is its identity stable?

One key insight to note here is that most people only really evaluate at the prompt layer, like asking "did I get the output I wanted?". This new stack makes explicit that failures at any layer propagate upward, and evaluation must be able to locate *which* layer failed, not just that the output is wrong.

Meta-Function: Harness Engineering *(Where and How to Do)*

Harness engineering governs the infrastructure, tooling, and execution environment in which the stack operates. It is the second meta-function, and not a layer of cognitive concern, but the platform on which all cognitive concerns are implemented.

If the evaluation asks "how do we know it's working?", the harness asks "where does all of this actually run?"

What It Governs

Harness engineering concerns itself with:

- **Execution platforms** — What IDE, CLI tool, API, or orchestration framework actually runs the AI interaction? (Cursor, Codex, VS Code + Copilot, Claude Code, custom API pipelines, LangChain, etc.)
- **Tool integration** — What external tools does the AI have access to? File systems, databases, web search, code execution, deployment pipelines.
- **Orchestration** — How are multi-step tasks structured? Sequential agents, parallel workers, supervisor patterns, human-in-the-loop checkpoints.
- **Session management** — How is conversation history maintained? How are long tasks segmented? How do context windows get managed across interactions?
- **CI/CD and automation** — How do evaluation criteria get executed automatically? Test runners, linting pipelines, pre-commit hooks, deployment gates.
- **Configuration and reproducibility** — Can a working stack configuration be saved, shared, and reused? Project rules, system prompts, template libraries.

Why It's Not a Layer

Harness engineering doesn't change *what the AI thinks about*, but it changes *where the AI thinks*. Switching from Cursor to Claude Code, for example, doesn't alter your prompt, context, intent, judgment, or coherence concerns. It alters what tools are available, how context gets loaded, and how evaluation gets executed. Even remaining in a platform like Claude Code which is often updated by a company (Anthropic), means certain changes introduced may slightly affect how your workflow runs at each layer, and if you are not aware of the changes then you may get unreliable or unpredictable changes at any given scale. Ultimately, these are all real engineering decisions, but they're specifically infrastructure decisions, rather than cognitive ones.

The relationship between harness and the stack is the same as the relationship between a kitchen and a recipe. The recipe (the layers) defines what you're making. The kitchen (the harness) determines what equipment you have to make it with. A better kitchen doesn't change the recipe, but it changes what's *feasible* and what's *efficient*.

The Relationship Between Harness and Evaluation

Evaluation *runs through* the harness. When you set up a test suite, the tests themselves are evaluation engineering (what to check, what criteria to use, what thresholds matter). The test runner, the CI pipeline, the logging framework, those are the harness. Evaluation defines the *what* of observation, and the harness provides the *where* of execution.

This is why the diagram shows evaluation resting on and running through the harness frame, because they are tightly coupled in practice, even though they are categorically distinct.

User Interaction Model

The stack is not a one-shot configuration, but a continuous interaction between the user and the layers, mediated by evaluation.

Entry Point

The user begins at the bottom, with **prompt engineering**. Every AI interaction starts with an instruction. But before the first prompt is written, the user has typically already made harness decisions (which tool to use) and may have configured context (project files, documentation), intent (goals and priorities in system prompts or project rules), judgment (uncertainty protocols), and coherence (behavioral guidelines). The stack is configured bottom-up but often *set up* in parallel.

The Diagnostic Loop

When output is unsatisfactory, the natural instinct is to rewrite the prompt. This stack helps provide a more precise diagnostic:

1. **Evaluation** detects a problem, either autonomously (tests fail) or through human review (the output doesn't seem right).
2. **Diagnosis** identifies which layer failed. Was the instruction unclear (prompt)? Was information missing (context)? Were the wrong tradeoffs made (intent)? Was a risk ignored (judgment)? Is the behavior inconsistent with previous work (coherence)?
3. **Re-entry** happens at the specific layer that needs correction. Not always the prompt. Often the fix is adding a file to context, clarifying a priority, or defining an uncertainty protocol, none of which involve changing the prompt at all.

The resulting effect becomes **mode fluency**, or the skill of recognizing which layer is failing and intervening at the right level. It is arguably the core skill the AI-Human Engineering Stack is designed to develop. Most AI frustration comes from trying to fix context problems with better

prompts, or judgment problems with more context. This stack now hopefully gives you a better vocabulary for diagnosing correctly.

Harness Intervention

The user also sets up and adjusts the harness, typically at the beginning of a project and then when capabilities need to change. Choosing a new tool, integrating a new data source,or setting up a test pipeline, these are all harness interventions that affect what's *possible* across all layers.

Boundaries of the Stack: Why These Five Layers

A taxonomy, however, is only as strong as its exclusions. The five layers were selected because each one introduces a **categorically distinct question** that cannot be reduced to any other layer. Several candidates were considered and found to collapse into existing layers:

Memory Engineering → Context Engineering. Memory, like persistent knowledge, session history, and learned preferences, is a delivery mechanism for context. Whether information arrives via file upload, retrieval system, or memory store, the engineering question is the same: what should the AI know? Memory doesn't add a new question; it adds a new answer to an existing one.

Persona / Role Engineering → Coherence Engineering. Giving the AI a character, voice, or professional identity is one expression of coherence. The deeper question, such as if the AI maintains a stable, principled identity across time, is what coherence governs. Persona is the surface, while coherence is the structure.

Tool / Function Engineering → Harness Engineering. Deciding what tools the AI can access, like file systems, APIs, code execution, and web search to name a few, is an infrastructure concern. It changes what's possible, not what the AI should think about. Tools are part of the execution environment, but not a cognitive layer.

Safety Engineering → Judgment Engineering. Preventing harmful outputs, avoiding hallucination, and maintaining truthfulness, these are domains in which judgment operates. The categorical question is the same: when should the AI doubt itself, and what should it do about that doubt? Safety is a high-stakes instance of judgment, not a separate kind of thinking.

Output / Format Engineering → Prompt Engineering. Specifying that the output should be JSON, markdown, a numbered list, or a particular document structure is an instruction. It tells the AI what to do. Format constraints are prompt constraints expressed at the output boundary.

Workflow / Orchestration Engineering → Harness Engineering. Multi-agent systems, swarms, sequential pipelines, supervisor patterns, human checkpoints, these are all execution

architectures. They determine *how the work is structured*, which is an infrastructure concern. The cognitive concerns at each step within the workflow are still governed by the five layers.

These candidates are real concerns, and practitioners who use these terms are pointing at something genuine. The claim here is not that they don't matter, but rather that they don't require their own layer in the stack because they don't introduce a question that isn't already asked by an existing layer or meta-function. A good taxonomy is ideally minimal, with every component being necessary, and no component being redundant.

An astute reader might wonder why we only specified 5 hierarchical layers, and not for example 15. Each layer in the stack addresses a problem at a progressively higher level of abstraction. But abstraction has diminishing returns. Beyond coherence engineering, the next hypothetical layer would address the question: "*How do we ensure that the entire human-AI civilization, including the humans who design the intent specs, the organizations that deploy the agents, and the societies affected by the outcomes, remains aligned with human flourishing?*" At that level, the problem is no longer engineering. It is politics, ethics, and philosophy. These are real and important problems, but they are not engineering disciplines in any recognizable sense. They belong to democratic governance, moral philosophy, and institutional design.

The DIKW hierarchy mentioned earlier in this document has also been criticized for implying an infinite regress: if wisdom evaluates knowledge, what evaluates wisdom? The same potential regress exists here: if judgment monitors intent, what monitors judgment? If coherence governs individual agents, what governs the coherence framework itself?

The answer is that the regress is terminated by human governance. At some point in every system (no matter how sophisticated its autonomous capabilities) there must be a human (or a human institution) that says: "This is what we value. This is what we are willing to be held accountable for. This is the foundation on which everything else rests." That foundation cannot be automated without circularity, because the question "what should we ultimately value?" is constitutive of human agency itself.

At face value this might seem like some inherent limitation of AI systems, but it is actually a structural feature of any system that acts on behalf of human purposes. The same limit applies to human organizations: no management framework, no governance structure, no set of rules and processes can substitute for the foundational act of deciding what matters.

Each layer requires the previous layers to be at least partially solved before it becomes the binding constraint. Right now, most organizations have not solved context engineering, let alone intent engineering. Judgment and coherence engineering are intellectually important but practically premature for most teams.

The progression has a natural deployment order, and attempting to skip layers is counterproductive. Building sophisticated judgment mechanisms for a system that lacks intent specification is solving a problem that does not yet exist. Building coherence frameworks for

organizations that have not deployed their first autonomous agent is architecture without a building.

The practical advice we can give is: identify which layer is your current binding constraint, invest there, and prepare for the next layer. For most individuals in early 2026, that means mastering context engineering and beginning to develop intent engineering. The other layers will become relevant as the stack beneath them matures.

Relationship to Specifying Work for AI

The AI-Human Engineering Stack provides the **conceptual architecture**, such as what dimensions of AI behavior exist and how they relate. The companion document, *Specifying Work for AI*, provides the **operational workflow**, or how to translate these dimensions into written specifications that direct AI work effectively.

A well-written specification touches every layer of the stack:

- It defines the task clearly (**prompt**)
- It identifies what information the AI needs and how to structure it (**context**)
- It articulates goals, priorities, and tradeoffs (**intent**)
- It sets boundaries for uncertainty, risk, and escalation (**judgment**)
- It establishes behavioral principles that persist across the work (**coherence**)

The specification *is* the stack made concrete for a particular task. Readers who want to move from understanding these layers to practicing them should consult that document for detailed guidance on the writing process itself.

Conclusion

The AI-Human Engineering Stack is a framework for thinking clearly about what it means to direct AI work. Its core claim is that there are five categorically distinct layers of cognitive concern, two meta-functions that operate across them, and a user interaction model that ties diagnosis to intervention.

The layers are ordered by dependency: each rests on all layers beneath it. They are ordered by abstraction: each is harder to see, harder to specify, and harder to evaluate than the one below. And they are ordered by neglect: most current practice concentrates on the bottom two layers, leaving intent, judgment, and coherence to accident and defaults.

The gap between people who use AI effectively and people who struggle is rarely about prompt skill. It is almost always about the invisible layers, the ones this stack is designed to make visible.

The specific LLM models will change over time, and related tools will evolve. The specific techniques of prompt engineering in 2026 will be as dated as web development practices from 2010 by the time another decade passes. But this stack, and the conceptual architecture of how raw instruction becomes coherent collective action, *that*, we suspect will endure, because it describes a structure that is not specific to any model, any tool, or any era.

Prompt engineering is, after all, the skill of clear communication. Context engineering is, after all, the skill of information management. Intent engineering is, after all, the skill of purpose articulation. Judgment engineering is, after all, the skill of knowing the limits of your own knowledge. Coherence engineering is, after all, the skill of making many autonomous parts work as one.

These are not really AI skills. They are intelligence skills, and are applicable to any collaboration between any entities capable of autonomous action, whether those entities are language models, human employees, organizational departments, or entire institutions or countries.

The AI-Human Engineering Stack is sort of a theory of collaboration itself, instantiated for the specific and unprecedented case where one class of collaborator has immense capability, zero shared background, and no innate sense of purpose. That case happens to be the defining challenge of our current technological moment as of 2026. But the principles are quite likely permanent.

So, we encourage you to learn the stack, and to apply it where you are. The layer that is your constraint today will not necessarily be your constraint tomorrow, but the one above it will be. And that is a clear direction of self-improvement for your skills.